PLAYER 1

250/300

아닛?! 이렇게 쉽다니!!

슬리드 5원칙

게임 시작하기

PLAYER 1

250/300

관효 전사

WARRIOR

형욱 마법사

MAGE

윤혜 힐러

HEALER

희원 궁수

ARCHER

1. 단일 책임

2. 개방 폐쇄

3. 리스코프 치환

4. 인터페이스 분리

5. 의존 역전

PLAYER 1

250/300

```java
class Warrior {

    // 공격
    void attack() {
        System.out.println("SwordAttack!");
    }

    // 방어
    void defend() {
        System.out.println("Block!");
    }

    // 이동
    void move(int x, int y) {
        System.out.println("Move to " + x + ", " + y);
    }

    // 아이템 줍기
    void pickItem(String item) {
        System.out.println(item + " picked up");
    }

    // 인벤토리 관리
    void openInventory() {
        System.out.println("Inventory opened");
    }
}
```

PLAYER 1

250/300

```
class Warrior {

    // 공격
    void attack() {
        System.out.println("SwordAttack!");
    }

    // 방어
    void defend() {
        System.out.println("Block!");
    }

    // 이동
    void move(int x, int y) {
        System.out.println("Move to " + x + ", " + y);
    }

    // 아이템 줍기
    void pickItem(String item) {
        System.out.println(item + " picked up");
    }

    // 인벤토리 관리
    void openInventory() {
        System.out.println("Inventory opened");
    }
}
```

SRP 위반!!!

PLAYER 1

250/300

```java
class Warrior {

    private final Attack attack;
    private final Defense defense;
    private final Movement movement;
    private final Inventory inventory;

    Warrior(Attack attack, Defense defense,
            Movement movement, Inventory inventory) {
        this.attack = attack;
        this.defense = defense;
        this.movement = movement;
        this.inventory = inventory;
    }

    void attack() {
        attack.execute();
    }

    void defend() {
        defense.block();
    }

    void move(int x, int y) {
        movement.move(x, y);
    }

    void pickItem(String item) {
        inventory.pickItem(item);
    }

    void openInventory() {
        inventory.openInventory();
    }
}
```

```java
class Attack {
    void execute() {
        System.out.println("SwordAttack!");
    }
}
```

```java
class Defense {
    void block() {
        System.out.println("Block!");
    }
}
```

```java
class Movement {
    void move(int x, int y) {
        System.out.println("Move to " + x + ", " + y);
    }
}
```

```java
class Inventory {
    void pickItem(String item) {
        System.out.println(item + " picked up");
    }

    void openInventory() {
        System.out.println("Inventory opened");
    }
}
```

PLAYER 1

250/300

```
class Warrior {

    private final Attack attack;
    private final Defense defense;
    private final Movement movement;
    private final Inventory inventory;

    Warrior(Attack attack, Defense defense,
            Movement movement, Inventory inventory) {
        this.attack = attack;
        this.defense = defense;
        this.movement = movement;
        this.inventory = inventory;
    }

    void attack() {
        attack.execute();
    }

    void defend() {
        defense.block();
    }

    void move(int x, int y) {
        movement.move(x, y);
    }

    void pickItem(String item) {
        inventory.pickItem(item);
    }

    void openInventory() {
        inventory.openInventory();
    }
}
```

```
class Attack {
    void execute() {
        System.out.println("SwordAttack!");
    }
}
```

```
class Defense {
    void block() {
        System.out.println("Block!");
    }
}
```

```
class Movement {
    void move(int x, int y) {
        System.out.println("Move to " + x + ", " + y);
    }
}
```

```
class Inventory {
    void pickItem(String item) {
        System.out.println(item + " picked up");
    }

    void openInventory() {
        System.out.println("Inventory opened");
    }
}
```

OK!

PLAYER 1

250/300

1. 단일 책임

2. 개방 폐쇄

3. 리스코프 치환

4. 인터페이스 분리

5. 의존 역전

PLAYER 1

250/300

```
class Mage {

    void castSpell(String spellType) {
        if (spellType.equals("FIRE")) {
            System.out.println("FireBall!");
        } else if (spellType.equals("ICE")) {
            System.out.println("IceBlast!");
        } else if (spellType.equals("LIGHTNING")) {
            System.out.println("LightningStrike!");
        }
    }
}
```

PLAYER 1

250/300

```
class Mage {

    void castSpell(String spellType) {
        if (spellType.equals("FIRE")) {
            System.out.println("FireBall!");
        } else if (spellType.equals("ICE")) {
            System.out.println("IceBlast!");
        } else if (spellType.equals("LIGHTNING")) {
            System.out.println("LightningStrike!");
        }
    }
}
```

OCP 위반!!!

PLAYER 1

250/300

```
interface Spell {
    void cast();
}
```

```
class Mage {

    private final Spell spell;

    Mage(Spell spell) {
        this.spell = spell;
    }

    void castSpell() {
        spell.cast();
    }
}
```

```
class FireSpell implements Spell {
    public void cast() {
        System.out.println("FireBall!");
    }
}
```

java

```
class IceSpell implements Spell {
    public void cast() {
        System.out.println("IceBlast!");
    }
}
```

```
class LightningSpell implements Spell {
    public void cast() {
        System.out.println("LightningStrike!");
    }
}
```

```java
interface Spell {
    void cast();
}
```

```java
class Mage {

    private final Spell spell;

    Mage(Spell spell) {
        this.spell = spell;
    }

    void castSpell() {
        spell.cast();
    }
}
```

```java
class PoisonSpell implements Spell {
    public void cast() {
        System.out.println("PoisonCloud!");
    }
}
```

```java
Mage mage = new Mage(new PoisonSpell());
mage.castSpell();
```

PLAYER 1

250/300

```java
interface Spell {
    void cast();
}
```

```java
class Mage {

    private final Spell spell;

    Mage(Spell spell) {
        this.spell = spell;
    }

    void castSpell() {
        spell.cast();
    }
}
```

```java
class PoisonSpell implements Spell {
    public void cast() {
        System.out.println("PoisonCloud!");
    }
}
```

```java
Mage mage = new Mage(new Poiso
mage.castSpell();
```

OK!

PLAYER 1

250/300

1. 단일 책임

2. 개방 폐쇄

3. 리스코프 치환

4. 인터페이스 분리

5. 의존 역전

PLAYER 1

250/300

```java
public class Character {
    public void attack() {
        System.out.println("공격합니다.");
    }
}
```

```java
class Healer extends Character {

    @Override
    void attack() {
        throw new UnsupportedOperationException();
    }

    void heal() {
        // 힐 기능
    }
}
```

PLAYER 1

250/300

```java
class Healer extends Character {

    @Override
    void attack() {
        throw new UnsupportedOperationException();
    }

    void heal() {
        // 힐 기능
    }
}
```

```java
Character a = new Warrior();
a.attack(); // OK!

Character b = new Mage();
b.attack(); // OK!
```

```java
Character c = new Healer();
c.attack(); // 런타임 에러 발생
```

PLAYER 1

250/300

```java
class Healer extends Character {

    @Override
    void attack() {
        throw new UnsupportedOperationException();
    }

    void heal() {
        // 힐 기능
    }
}
```

```java
Character a = new Warrior();
a.attack(); // OK!

Character b = new Mage();
b.attack(); // OK!
```

```java
Character c = new Healer();
c.attack(); // 런타임 에러 발생
```

LSP 위반!!!

PLAYER 1

250/300

```java
public class Healer extends Character {

    @Override
    public void attack() {
        System.out.println("힐러는 공격할 수 없습니다.");
    }

    public void heal() {
        System.out.println("회복합니다.");
    }
}
```

```java
Character c = new Healer();
c.attack();  // 정상 작동
```

PLAYER 1

250/300

```java
public class Healer extends Character {

    @Override
    public void attack() {
        System.out.println("힐러는 공격할 수 없습니다.");
    }

    public void
        System.
    }
}
```

OK..?

```java
Character c = new Healer();
c.attack();  // 정상 작동
```

Canva

Canva

PLAYER 1

250/300

1. 단일 책임

2. 개방 폐쇄

3. 리스코프 치환

4. 인터페이스 분리

5. 의존 역전

PLAYER 1

250/300

```
abstract class Character {
    int hp;
}
```

```
nterface CharacterAction {
    void attack();
    void heal();
    void castSpell();
```

```
interface Attackable {
    void attack();
}
```

```
class Warrior extends Character implements Attackable {
    @Override
    public void attack() {
        System.out.println("전사가 공격합니다.");
    }
}
```

```
class Mage extends Character implements Attackable {

    private final Spell spell;

    Mage(Spell spell) {
        this.spell = spell;
    }

    @Override
    public void attack() {
        spell.cast();
    }
}
```

PLAYER 1

250/300

```
interface Healable {
    void heal();
}
```

```
class Healer extends Character implements Healable {

    private final Spell healSpell;

    Healer(Spell healSpell) {
        this.healSpell = healSpell;
    }

    @Override
    public void heal() {
        healSpell.cast();
    }
}
```

PLAYER 1

250/300

1. 단일 책임

2. 개방 폐쇄

3. 리스코프 치환

4. 인터페이스 분리

5. 의존 역전

PLAYER 1

250/300

```java
class NormalArrow {
    void shoot() {
        System.out.println("일반 화살 발사!");
    }
}
```

```java
class Archer extends Character implements Attackable {

    private NormalArrow arrow = new NormalArrow();

    @Override
    public void attack() {
        arrow.shoot();
    }
}
```

PLAYER 1

250/300

```
class NormalArrow {
    void shoot() {
        System.out.println("일반 화살 발사!");
    }
}
```

```
class Archer extends Character implements Attackable {

    private NormalArrow arrow = new NormalArrow();

    @Override
    public void attack() {
        arrow.shoot();
    }
}
```

DIP 위반!!!

PLAYER 1

250/300

```java
interface Arrow {
    void shoot();
}
```

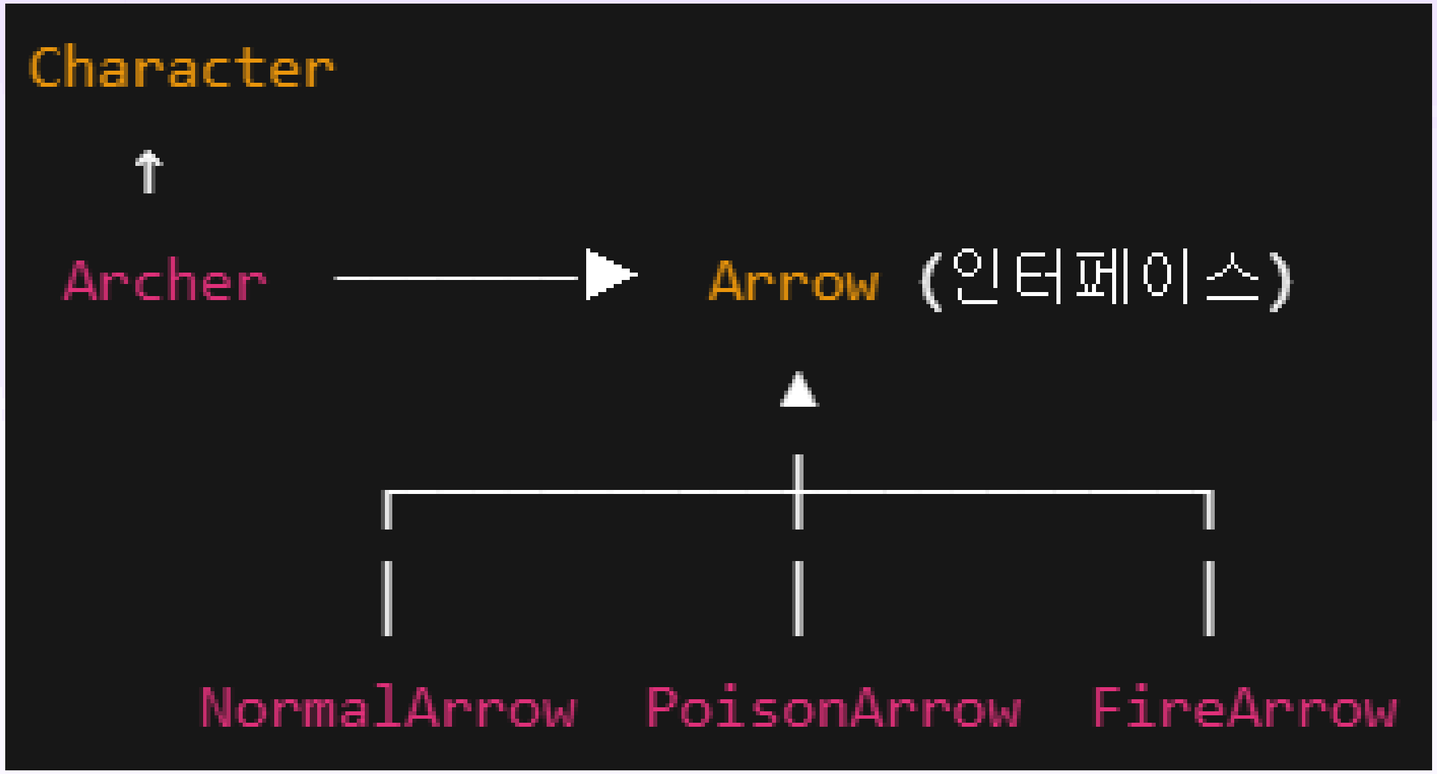
```java
class NormalArrow implements Arrow {
    public void shoot() {
        System.out.println("일반 화살 발사!");
    }
}
```

```java
class PoisonArrow implements Arrow {
    public void shoot() {
        System.out.println("독 화살 발사!");
    }
}
```

```java
class FireArrow implements Arrow {
    public void shoot() {
        System.out.println("불 화살 발사!");
    }
}
```

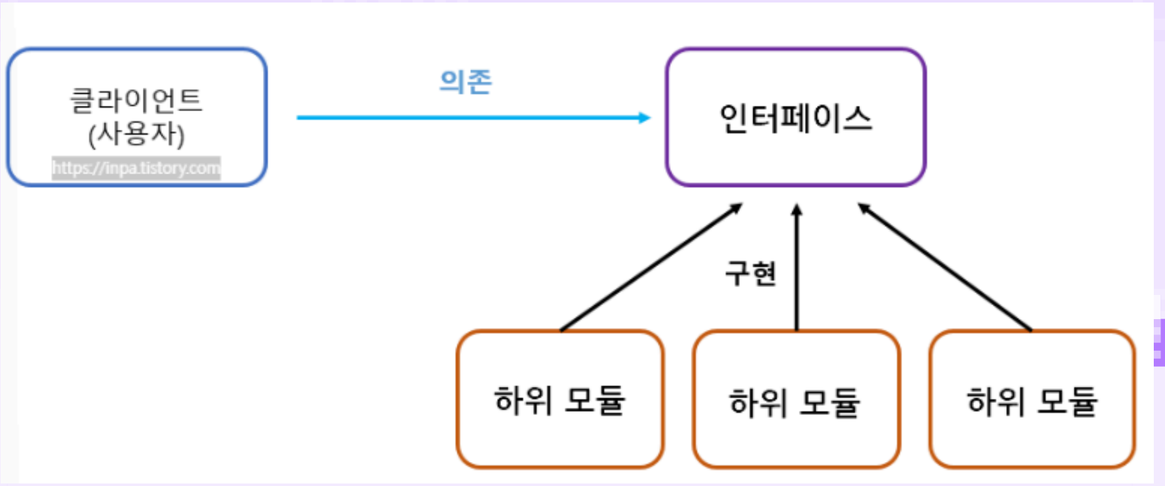
PLAYER 1

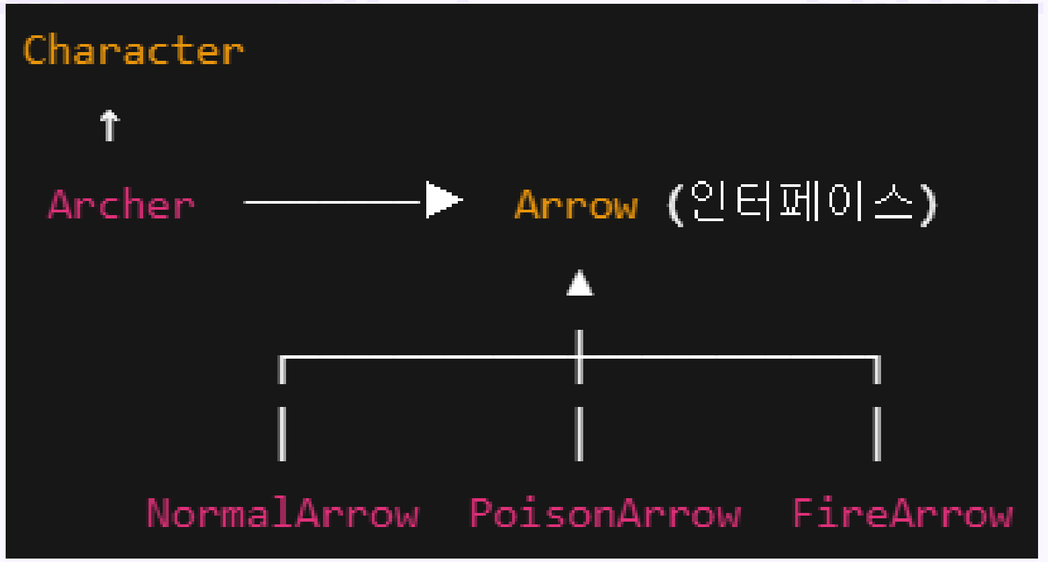
250/300



PLAYER 1

250/300





PLAYER 1

250/300

```java
class Archer extends Character implements Attackable {

    private final Arrow arrow;

    Archer(Arrow arrow) {
        this.arrow = arrow;
    }

    @Override
    public void attack() {
        arrow.shoot();
    }
}
```

```java
Archer archer1 = new Archer(new NormalArrow());
archer1.attack();

Archer archer2 = new Archer(new PoisonArrow());
archer2.attack();
```

PLAYER 1

250/300

```java
class Archer extends Character implements Attackable {

    private final Arrow arrow;

    Archer(Arrow arrow) {
        this.arrow = arrow;
    }

    @Override
    public void attack() {
        arrow.shoot();
    }
}
```

```java
Archer archer1 = new Archer(new NormalArrow());
archer1.attack();

Archer archer2 = new Archer(new PoisonArrow());
archer2.attack();
```

PLAYER 1

250/300

감사합니다!